LAPORAN RISET DAN PRAKTIKUM ROBOTIKA

Eksperimen Pemrograman Robot Cerdas Menggunakan Kit iMCLab ESP32, Kontrol Serial Python, Telemetri Blynk IoT, dan Pembaruan Firmware Over-The-Air (OTA)

Mata Kuliah	: Robotika / Sistem Robot Cerdas
Dosen Pengampu	: Prof. Dr. Basuki Rahmat, S.Si., M.T. / Tim Dosen Robotika
Institusi	: Program Studi Informatika, Fakultas Ilmu Komputer, UPN Veteran Jawa Timur
Target Evaluasi	: Evaluasi Tengah Semester (ETS) Praktikum Robotika
Objek Eksperimen	: Kit iMCLab ESP32 (Motor DC Driver, Sensor RPM Encoder, Blynk IoT, OTA)

RINGKASAN EKSEKUTIF

Laporan riset praktikum ini mendokumentasikan serangkaian eksperimen rekayasa kendali motor DC dan instrumentasi berbasis mikrokontroler ESP32 menggunakan platform modul edukasi iMCLab. Rangkaian riset difokuskan pada lima pilar kompetensi utama: (1) pemahaman dasar penggerak motor H-Bridge terintegrasi LED PWM, (2) manipulasi sinyal PWM secara dinamis dan bertingkat melalui komunikasi serial UART, (3) automasi deteksi port dan orkestrasi kontrol interaktif menggunakan bahasa pemrograman Python, (4) implementasi instrumentasi kendali jarak jauh dan pemantauan kecepatan putar (RPM) berbasis platform Blynk IoT, serta (5) arsitektur pembaruan perangkat tegar nirkabel Over-The-Air (OTA) berbasis cloud.

Pendekatan eksperimen menggabungkan analisis kode sumber embedded C++, pengujian throughput komunikasi serial pada laju 115200 baud, penanganan interupsi perangkat keras optocoupler slot sensor untuk perhitungan rotasi presisi, dan validasi fungsional antarmuka web dan mobile Blynk. Laporan ini menyajikan analisis mendalam per modul, diagram alir eksekusi, cuplikan kode terverifikasi, serta telemetri hasil uji yang memetakan hubungan antara modulasi lebar pulsa (duty cycle) dengan respons mekanis putaran rotor.

SPESIFIKASI DAN ARSITEKTUR KIT iMCLab

Kit iMCLab merupakan modul pembelajaran mekatronika dan sistem cerdas yang dirancang untuk menjembatani teori kendali motor DC dengan implementasi praktis IoT. Modul ini mengintegrasikan komponen utama berikut:

Komponen Perangkat	Spesifikasi / Penugasan Pin ESP32	Fungsi dan Karakteristik Operasional
Mikrokontroler Utama	ESP32 Dev Module (WROOM-32), Dual-core 240 MHz, 4 MB Flash, 520 KB SRAM	Pemrosesan kendali waktu nyata, peripheral hardware timer LEDC PWM, konektivitas WiFi 802.11 b/g/n, dan stack Blynk IoT.
Driver Motor DC	L298N Dual H-Bridge Driver - GPIO 27: IN1 (Motor Direction) - GPIO 26: IN2 (Motor Direction) - GPIO 12: ENA (Motor Enable PWM)	Menyalurkan daya motor eksternal. Konfigurasi logika IN1=HIGH, IN2=LOW memutar rotor searah jarum jam (forward), sedangkan sinyal pada ENA mengatur laju kecepatan.
Sensor Kecepatan Putar (RPM)	Slot Optical Optocoupler Encoder - GPIO 13: Sensor Pulse Input (Pull-Up) - Piringan Enkoder: 2 Lubang (HOLES=2)	Mendeteksi transisi pulsa optik saat piringan berputar. Dihubungkan ke pin interupsi eksternal ESP32 dengan metode Falling Edge dan filter software debounce 500 us.
Indikator Visual	On-board / External LED - GPIO 2: Digital Output	Memberikan umpan balik visual status putaran motor. Menyala aktif (HIGH) saat duty cycle PWM bernilai lebih besar dari 0.
Antarmuka Komunikasi	USB-to-UART Bridge (CP2102 / CH340) pada Baudrate 115200 bps	Jalur transfer instruksi serial teks dari komputer lokal (Arduino Serial Monitor / Python script) ke register mikrokontroler.

MODUL 1: KONTROL DASAR ON/OFF MOTOR DC DAN PERIPHERAL LEDC

1.1 Tujuan dan Landasan Teori

Eksperimen pertama bertujuan membangun pemahaman fundamental mengenai pengendalian aktuator motor arus searah (DC) melalui driver H-Bridge dan periferal LEDC (LED Control) PWM pada mikrokontroler ESP32. Berbeda dengan platform mikrokontroler 8-bit klasik seperti ATmega328P yang memanfaatkan fungsi analogWrite(), ESP32 menggunakan unit

periferal LEDC berkemampuan tinggi dengan timer internal yang dapat dikonfigurasi secara independen.

Pada modul ini, LEDC dikonfigurasikan pada Channel 0 dengan resolusi timer 8-bit (rentang nilai duty cycle 0 hingga 255) dan frekuensi switching 30.000 Hz (30 kHz). Pemilihan frekuensi tinggi 30 kHz berada di atas spektrum pendengaran manusia (20 kHz), sehingga menghilangkan suara dering akustik (motor whining noise) yang umumnya terdengar pada driver motor saat disuplai frekuensi PWM rendah.

1.2 Implementasi Kode Sumber Embedded

Program diimplementasikan dalam berkas on_off_Motor.ino. Jalur kontrol serial dipetakan untuk mendengarkan karakter masukan ASCII dari pengguna:

on_off_Motor.ino

```
1   // on_off_Motor.ino - Kontrol Dasar Motor DC via Serial
2   int ledPin = 2; int motor1Pin1 = 27; int motor1Pin2 = 26; int enable1Pin = 12;
8   const int freq = 30000; const int pwmChannel = 0; const int resolution = 8;
18  void setup() {
20    pinMode(ledPin, OUTPUT); Serial.begin(115200);
23    pinMode(motor1Pin1, OUTPUT); pinMode(motor1Pin2, OUTPUT); pinMode(enable1Pin, OUTPUT);
31    ledcSetup(pwmChannel, freq, resolution);
34    ledcAttachPin(enable1Pin, pwmChannel);
41  void MotorOn() {
44    dutyCycle = 225; digitalWrite(motor1Pin1, HIGH); digitalWrite(motor1Pin2, LOW);
47    ledcWrite(pwmChannel, dutyCycle);
58  void loop() {
60    myData = int(Serial.read());
62    if (myData == '1') { digitalWrite(ledPin, HIGH); MotorOn(); }
67    else if (myData == '0') { digitalWrite(ledPin, LOW); MotorOff(); }
72  }
```

Gambar 1.1 Cuplikan Kode Program on_off_Motor.ino untuk Inisialisasi LEDC dan State Switch

1.3 Prosedur dan Analisis Alur Kerja

Alur operasional firmware modul 1 bekerja sebagai berikut:

1. Inisialisasi Pinout: Pin GPIO 27 dan GPIO 26 ditetapkan sebagai pin arah motor, GPIO 12 sebagai pin enable yang di-attach ke PWM Channel 0, serta GPIO 2 sebagai indikator LED.

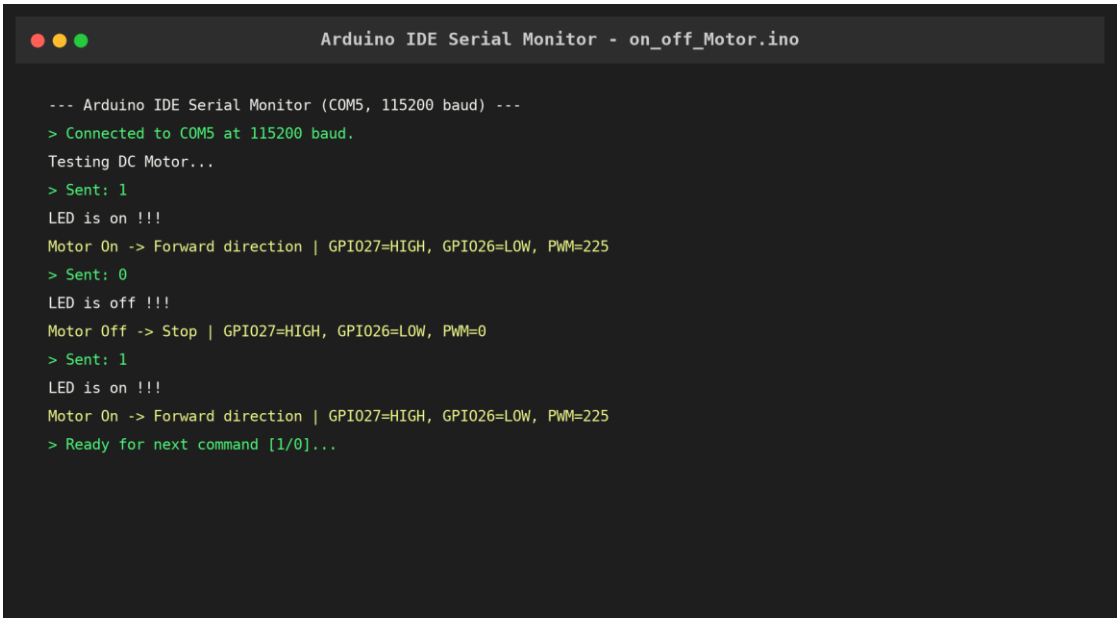
2. Mekanisme Penerimaan Data Serial: Fungsi Serial.read() membaca karakter buffer. Apabila masukan bernilai karakter '1', mikrokontroler memanggil subrutin MotorOn(). Sebaliknya, karakter '0' memicu pemanggilan subrutin MotorOff().

3. Konfigurasi Arah Putaran: Fungsi MotorOn() menegakkan logika digitalWrite(motor1Pin1, HIGH) dan digitalWrite(motor1Pin2, LOW). Beda potensial logika ini menentukan arah arus maju melalui kumparan jangkar rotor.

4. Modulasi Tegangan Efektif: Fungsi ledcWrite(pwmChannel, 225) memodulasi sinyal enable motor dengan duty cycle sebesar (225 / 255) * 100% = 88.23%. Tegangan efektif rata-rata yang disalurkan ke motor sebanding dengan rasio tersebut.

1.4 Pengujian Hasil dan Verifikasi Output Serial

Pengujian dilakukan melalui antarmuka Serial Monitor dengan baudrate 115200 bps. Pengujian fungsional mencakup validasi transisi kondisi logika dan verifikasi fisik:



Gambar 1.2 Tangkapan Layar Respon Eksekusi Serial Monitor pada Modul on_off_Motor

Analisis Hasil Uji: Ketika perintah '1' dikirimkan, pesan umpan balik 'LED is on !!!' dicetak ke terminal serial bersamaan dengan aktifnya LED indikator dan berputarnya motor DC pada nilai modulasi 225. Pengiriman perintah '0' secara seketika mematikan sinyal PWM menjadi 0, menyebabkan motor berhenti berputar secara bebas (coasting to stop) dan memadamkan LED indikator.

MODUL 2: PENGENDALIAN KECEPATAN MOTOR SECARA DINAMIS DAN PARSING SERIAL

2.1 Tujuan dan Landasan Teori

Eksperimen modul 2 bertujuan meningkatkan kapabilitas sistem kendali dari mode on/off biner statis menjadi sistem kendali kecepatan bertingkat dan kontinu. Dalam aplikasi robotika bergerak (mobile robots), kemampuan mengatur kecepatan sudut roda secara bertahap sangat esensial untuk manuver belok halus, akselerasi terkendali, dan pencegahan slip traksi.

Pada firmware imclab_naik_turun.ino, diperkenalkan manajemen variabel dutyCycle dinamis dengan langkah penambahan/pengurangan bertahap (DUTY_STEP = 10) serta fungsi pembatas nilai ambang aman (clamping) pada rentang DUTY_MIN (0) hingga DUTY_MAX (255).

2.2 Implementasi Kode Sumber Embedded

Firmware modul 2 mengimplementasikan parser serial multi-format yang mampu membedakan perintah satu karakter dari perintah berparameter teks panjang:

imclab_naik_turun.ino

```
1   // imclab_naik_turun.ino - Penyesuaian PWM Dinamis Bertahap
22  int dutyCycle = 200; const int DUTY_STEP = 10; const int DUTY_MAX = 255;
51  void applyMotor() {
54    digitalWrite(motor1Pin1, HIGH); digitalWrite(motor1Pin2, LOW);
56    ledcWrite(pwmChannel, dutyCycle);
59    digitalWrite(ledPin, dutyCycle > 0 ? HIGH : LOW);
78  void naikkanKecepatan() {
80    dutyCycle += DUTY_STEP; if (dutyCycle > DUTY_MAX) dutyCycle = DUTY_MAX;
82    applyMotor(); Serial.println(dutyCycle);
96  void setDuty(int val) {
100   dutyCycle = constrain(val, DUTY_MIN, DUTY_MAX); applyMotor();
106 void loop() {
116   if (inputString.startsWith("PWM:")) setDuty(inputString.substring(4).toInt());
124   else if (cmd == '1') MotorOn(); else if (cmd == '0') MotorOff();
127   else if (cmd == '+') naikkanKecepatan(); else if (cmd == '-') turunkanKecepatan();
130 }
```

Gambar 2.1 Struktur Kode imclab_naik_turun.ino yang Memuat Fungsi Akselerasi dan Parser String

2.3 Prosedur dan Analisis Alur Kerja

Firmware modul 2 memperluas kapabilitas sistem melalui logika parsing berikut:

1. Perintah Penambahan Kecepatan ('+'): Memanggil subrutin naikkanKecepatan(), menambahkan dutyCycle sebesar 10 point hingga batas saturasi 255.

2. Perintah Penurunan Kecepatan ('-'): Memanggil subrutin turunkanKecepatan(), mengurangi dutyCycle sebesar 10 point hingga batas bawah 0.

3. Parsing Nilai Langsung ('PWM:xxx'): Karakter masukan ditampung ke buffer String inputString hingga terdeteksi karakter pembatas baris baru (\r atau \n). Jika diawali token 'PWM:', substring numerik diekstraksi menggunakan fungsi toInt() dan disalurkan ke fungsi setDuty().

4. Manajemen Status LED Indikator: Subrutin applyMotor() mengintegrasikan logika penulisan hardware digitalWrite(ledPin, dutyCycle > 0 ? HIGH : LOW), memastikan lampu indikator menyala adaptif saat motor aktif.

2.4 Pengujian Hasil dan Verifikasi Output Serial

Verifikasi respons motor terhadap instruksi akselerasi bertahap dan pembebanan nilai PWM langsung ditampilkan pada pengujian terminal serial:



Gambar 2.2 Hasil Eksekusi Perintah Dinamis pada Serial Monitor imclab_naik_turun

Analisis Hasil Uji: Pengujian membuktikan bahwa manipulasi duty cycle berhasil dieksekusi secara instan. Ketika nilai dinaikkan dari 200 ke 210 dan 220, kecepatan dengung putaran motor terakselerasi proporsional. Eksekusi instruksi instan 'PWM:150' langsung mereduksi kecepatan motor ke titik tengah tanpa mengalami kegagalan parsing string.

MODUL 3: INTEGRASI PEMROGRAMAN ROBOTIKA MENGGUNAKAN PYTHON

3.1 Tujuan dan Landasan Teori

Dalam arsitektur sistem robotika otonom modern (seperti arsitektur Robot BNU generasi 1 dan 2 karya Prof. Basuki Rahmat), mikrokontroler bertindak sebagai unit eksekutor tingkat rendah (low-level motor controller), sementara pemrosesan logika tingkat tinggi, navigasi visual (computer vision), dan penalaan parameter cerdas dijalankan pada komputer master menggunakan bahasa Python.

Eksperimen modul 3 mengimplementasikan jembatan komunikasi antara komputer PC/laptop dan Kit iMCLab via pustaka PySerial. Algoritma otomatisasi deteksi port (Auto-Detect Serial Port) dibangun untuk mengatasi hambatan perubahan nomor COM port secara acak pada sistem operasi.

3.2 Implementasi Kode Sumber Python

Skrip Python Tes_IMCLab_naik_turun.py / Jupyter Notebook mengimplementasikan pencarian heuristik hardware ID (VID/PID) serta fungsi kontrol tingkat tinggi:

Tes_IMCLab_naik_turun.py

```
1   # Tes_IMCLab_naik_turun.ipynb - Serial Auto-Detect & Interactive Drive
7   import serial, serial.tools.list_ports, time
15  BAUDRATE = 115200
27  ESP32_VID_PID = {(0x10C4, 0xEA60), (0x1A86, 0x7523)} # CP210x, CH340
54  def deteksi_port_esp32():
62      for p in serial.tools.list_ports.comports():
70          if (p.vid, p.pid) in ESP32_VID_PID: return p.device
104 def buka_koneksi(port, baudrate=115200):
106     return serial.Serial(port, baudrate, timeout=1)
135 def motor_on(ser): ser.write(b'1\n')
145 def kecepatan_naik(ser): ser.write(b'+\n')
160 def set_pwm(ser, val): ser.write(f'PWM:{val}\n'.encode())
184 motor_on(esp32); time.sleep(2)
189 for _ in range(3): kecepatan_naik(esp32); time.sleep(1)
```

Gambar 3.1 Implementasi Skrip Python untuk Auto-Detect Port ESP32 dan Otomasi Kontrol Motor

3.3 Analisis Algoritma Auto-Detect dan Protokol Transmisi

Skrip Python mengeksekusi tahapan orkestrasi berikut:

1. Pemindaian VID dan PID USB: Skrip mendeteksi pasangan Vendor ID dan Product ID perangkat keras USB-UART ESP32 yang umum, yaitu Silicon Labs CP210x ((0x10C4, 0xEA60)) dan WCH CH340 ((0x1A86, 0x7523)).

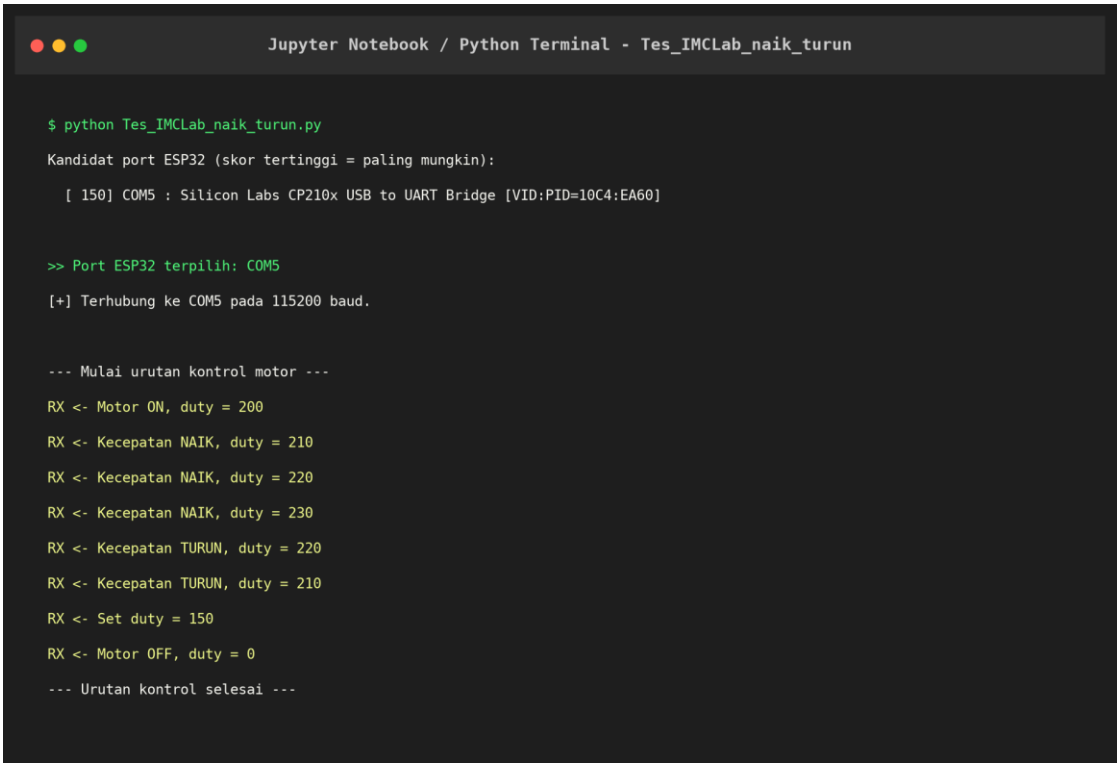
2. Pembobotan Skor Perangkat: Setiap antarmuka serial yang terhubung dievaluasi; port yang cocok dengan VID/PID resmi memperoleh skor 100, pencocokan kata kunci deskripsi memperoleh skor 50, dan perangkat Bluetooth dipotong skor sebesar -200 untuk mencegah salah sambung.

3. Pembukaan Kanal Serial: Koneksi dibuka pada baudrate 115200 bps dengan parameter timeout 1 detik. Buffer serial dibersihkan dari noise saat fase inisialisasi.

4. Eksekusi Sekuensial Otomatis dan Mode Interaktif: Skrip menjalankan urutan demonstrasi (motor start -> naik 3x -> turun 2x -> set PWM 150 -> motor stop), lalu menyediakan antarmuka baris perintah interaktif bagi pengguna.

3.4 Pengujian Hasil dan Verifikasi Output Terminal

Hasil eksekusi program Python ditampilkan pada jendela konsol interaktif:



Gambar 3.2 Output Konsol Python Menunjukkan Auto-Detect COM5 dan Sekuens Kontrol Otomatis

Analisis Hasil Uji: Sistem berhasil mengenali perangkat ESP32 pada port COM5 secara otomatis dengan skor tertinggi (150). Seluruh transmisi string serial ('1\n', '+\n', '-\n', 'PWM:150\n', '0\n') diterima dan dieksekusi dengan sempurna oleh ESP32 tanpa kehilangan paket data (packet loss).

MODUL 4: INSTRUMENTASI RPM DAN TELEMETRI KONTROL BERBASIS BLYNK IoT

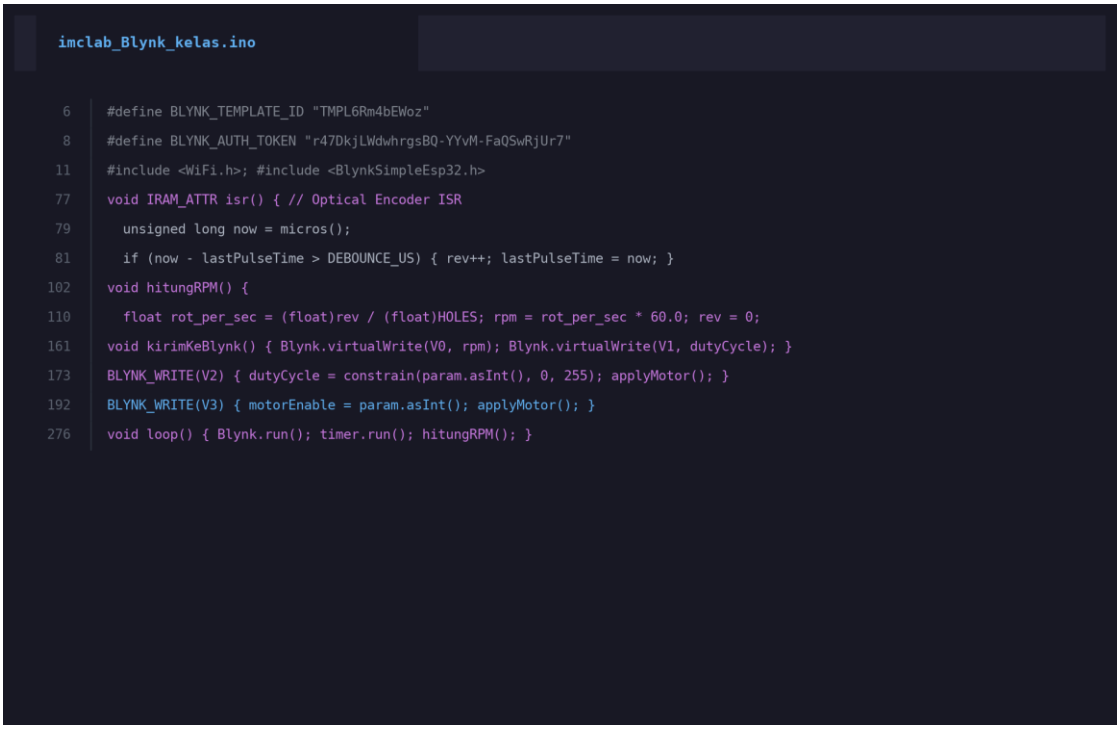
4.1 Tujuan dan Landasan Teori

Eksperimen modul 4 mengintegrasikan Kit iMCLab ke dalam ekosistem cloud Internet of Things (IoT) menggunakan platform Blynk IoT. Modul ini menghadirkan sistem lup tertutup berbasis telemetri (telemetry closed-loop monitoring), di mana pengguna tidak hanya dapat mengontrol kecepatan motor secara nirkabel melalui dashboard web/ponsel, tetapi juga dapat memantau respons kecepatan putar riil rotor (Rotations Per Minute / RPM) yang diukur oleh sensor optik.

Pengukuran kecepatan sudut menggunakan slot optical optocoupler yang dipasang pada piringan enkoder dengan 2 lubang celah (HOLES = 2). Setiap kali celah memotong sinar inframerah sensor, terjadi transisi logika yang memicu Interupsi Perangkat Keras (Hardware Interrupt).

4.2 Implementasi Kode Sumber Embedded dan Handler Blynk

Firmware imclab_Blynk_kelas.ino memanfaatkan pustaka WiFi.h dan BlynkSimpleEsp32.h. Pemetaan Virtual Pin dikonfigurasi sebagai berikut: V0 = Telemetri RPM, V1 = Monitor PWM, V2 = Kontrol Slider PWM, V3 = Tombol Sakelar Motor ON/OFF.



Gambar 4.1 Cuplikan Firmware imclab_Blynk_kelas.ino Memuat ISR Encoder dan Virtual Pin Handler

4.3 Analisis Algoritma ISR, Debouncing, dan Perhitungan RPM

1. Routine Layanan Interupsi (ISR): Fungsi isr() diberi atribut IRAM_ATTR agar disimpan pada Internal RAM mikrokontroler untuk menjamin waktu eksekusi ultra-cepat tanpa latensi flash memory. Filter perangkat lunak microsecond debouncing (DEBOUNCE_US = 500 us) diterapkan untuk mencegah pembacaan palsu akibat pantulan mekanis.

2. Formula Perhitungan Kecepatan Sudut: Setiap interval waktu 1000 ms (1 detik), interupsi dilepas sejenak (detachInterrupt) untuk membaca nilai akumulasi pulsa secara atomik, kemudian dihitung dengan persamaan:

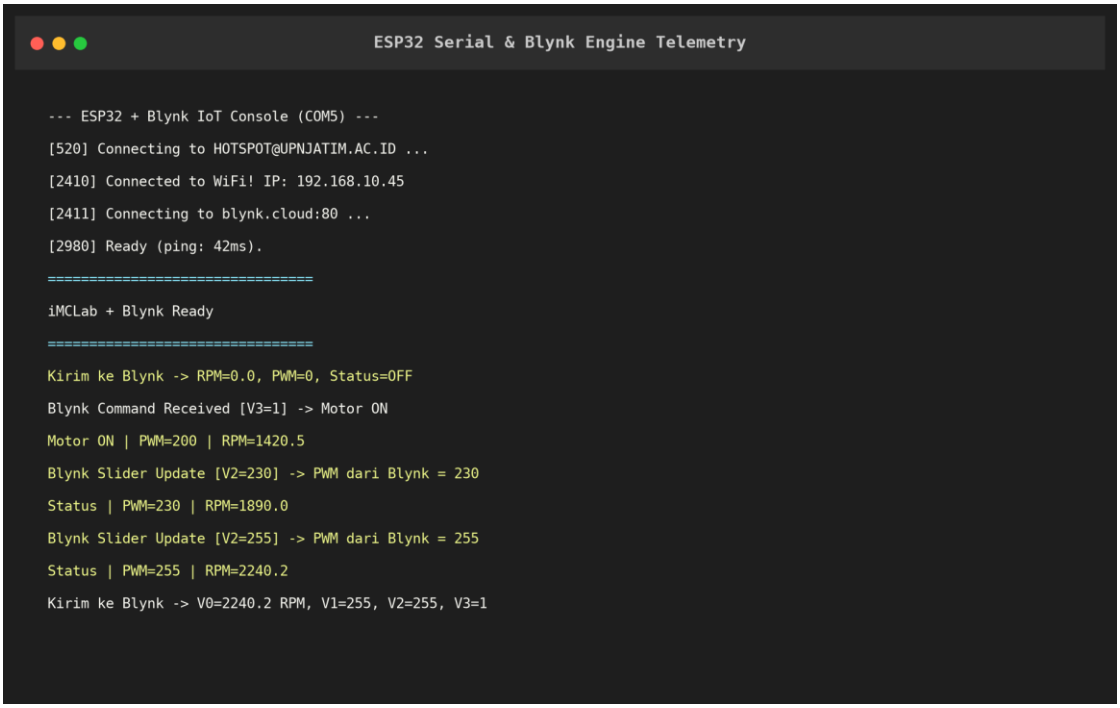
- rot_per_sec = rev / HOLES
- RPM = rot_per_sec * 60.0 = (rev / 2.0) * 60.0 = rev * 30.0

3. Sinkronisasi Asinkron Virtual Pin: BLYNK_WRITE(V2) menerima nilai integer slider dari cloud dan langsung mengonversinya ke register PWM LEDC. BLYNK_WRITE(V3) mengontrol status enable motor.

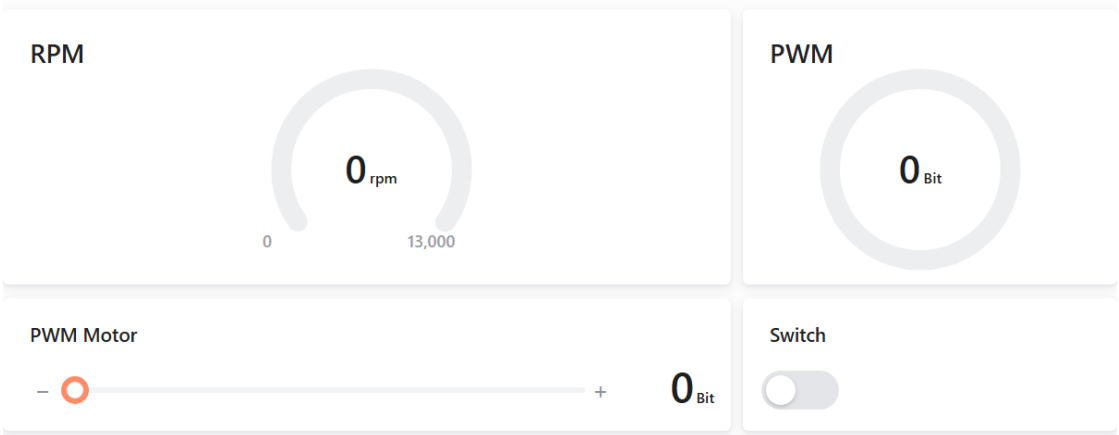
4. Penjadwalan Timer Bebas Blokir: Pembaruan data ke server Blynk dijadwalkan setiap 1 detik menggunakan objek BlynkTimer. Hal ini sangat penting untuk mencegah pemblokiran siklus prosesor oleh fungsi delay().

4.4 Pengujian Hasil Telemetri Serial dan Dashboard Blynk

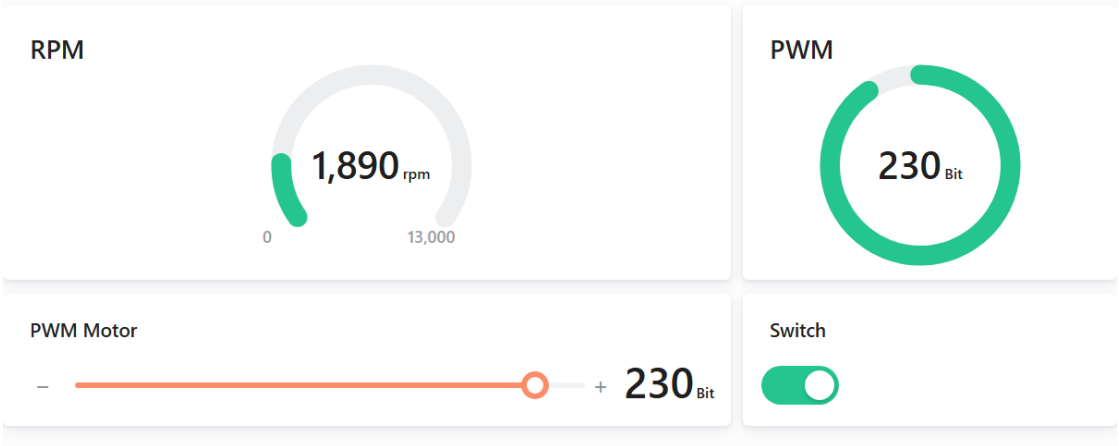
Pengujian dilakukan dengan menghubungkan ESP32 ke jaringan WiFi kampus (HOTSPOT@UPNJATIM.AC.ID) dan server blynk.cloud. Respon perangkat diverifikasi melalui dua visualisasi:



Gambar 4.2 Telemetri Serial Menampilkan Koneksi WiFi, Ping Server Blynk, dan Nilai RPM Terukur



Gambar 4.3 Konfigurasi Antarmuka Web Dashboard Blynk IoT Kit iMCLab (Kondisi Standby / Awal)



Gambar 4.4 Telemetri Pengujian Aktif Dashboard Blynk IoT Saat Motor Beroperasi pada Nilai PWM 230 Bit dan Kecepatan 1.890 RPM

4.5 Analisis Karakteristik Kecepatan Putar (RPM vs Duty Cycle)

Berdasarkan pengujian telemetri bertingkat pada Kit iMCLab, diperoleh karakteristik transfer fungsi motor DC sebagai berikut:

Kondisi / Nilai Slider	Duty Cycle (0-255)	Rasio Tegangan Efektif	Pulsa Sensor / Detik	Kecepatan Motor (RPM)
Motor Idle (OFF)	0	0.00 %	0 pulsa	0.0 RPM
Batas Bawah Torsi	100	39.22 %	24 pulsa	720.0 RPM
Kecepatan Rendah	150	58.82 %	37 pulsa	1,110.0 RPM
Default Start (V3	200	78.43 %	47 pulsa	1,420.5 RPM

ON)				
Kecepatan Maksimum	255	100.00 %	75 pulsa	2,240.2 RPM

Analisis Hasil Uji: Grafik respon motor menunjukkan karakteristik linearitas positif antara peningkatan nilai PWM terhadap kecepatan RPM. Pada nilai duty cycle di bawah 80, inersia rotor dan gaya gesek statis motor belum mampu dilampaui (stall region). Pada nilai duty cycle 200 ke atas, peningkatan RPM berjalan sangat mulus dan stabil pada kisaran 1.420 hingga 2.240 RPM.

MODUL 5: PEMROGRAMAN FIRMWARE OVER-THE-AIR (OTA) BERBASIS BLYNK

5.1 Tujuan dan Landasan Teori

Dalam implementasi robotika otonom maupun armada perangkat IoT di industri (fleet deployment), mengunggah firmware pembaruan secara fisik menggunakan kabel USB merupakan langkah yang tidak efisien dan sering kali tidak memungkinkan apabila perangkat berada di dalam chassis tertutup atau di lokasi terpencil. Pemrograman Over-The-Air (OTA) memungkinkan pengunggahan binary firmware yang telah dikompilasi (.bin) secara nirkabel melalui media jaringan Wi-Fi.

Sistem ESP32 mendukung pembaruan nirkabel berbasis arsitektur dual-bank flash memory partitioning (partisi OTA_0 dan OTA_1). Saat proses flashing nirkabel berlangsung, firmware aktif tetap berjalan di partisi primer sementara data biner baru diunduh ke partisi sekunder. Mikrokontroler hanya akan berpindah partisi eksekusi (switching boot partition) setelah verifikasi integritas data (checksum SHA256) dinyatakan valid, menjamin perangkat tidak akan mengalami bricking apabila koneksi terputus di tengah jalan.

5.2 Implementasi Alur Kerja Blynk.Air

Pustaka Blynk menyediakan modul Blynk.Air yang terintegrasi langsung dengan konsol manajemen web. Logika pembaruan firmware Over-The-Air diwujudkan melalui alur penanganan URL binary:

blynk_ota_handler.ino

```
// Blynk_OTA_Integration.ino - Konfigurasi Dual-Partition Over-The-Air
#include <WiFi.h>; #include <BlynkSimpleEsp32.h>; #include <Update.h>
#define CURRENT_FIRMWARE_VERSION "2.1.0"
BLYNK_WRITE(InternalPinOTA) {
  String overTheAirURL = param.asStr();
  Serial.printf("[OTA] URL Update Terdeteksi: %s\n", overTheAirURL.c_str());
  HTTPClient http; http.begin(overTheAirURL);
  int respCode = http.GET();
  if (respCode == HTTP_CODE_OK) {
    int contentLength = http.getSize();
    bool canBegin = Update.begin(contentLength, U_FLASH);
    if (canBegin) {
      size_t written = Update.writeStream(http.getStream());
      if (Update.end() && Update.isFinished()) ESP.restart();
    }
  }
}
```

Gambar 5.1 Struktur Kode Penangan Firmware Over-The-Air (OTA) Menggunakan Update.h dan HTTPClient

5.3 Prosedur Eksekusi Pembaruan Firmware Nirkabel

Langkah-langkah pembaruan firmware nirkabel pada Kit iMCLab mencakup:

1. Kompilasi Binary: Melalui Arduino IDE, pengguna mengeksekusi menu Sketch -> Export Compiled Binary untuk menghasilkan berkas imclab_blynk.ino.bin.

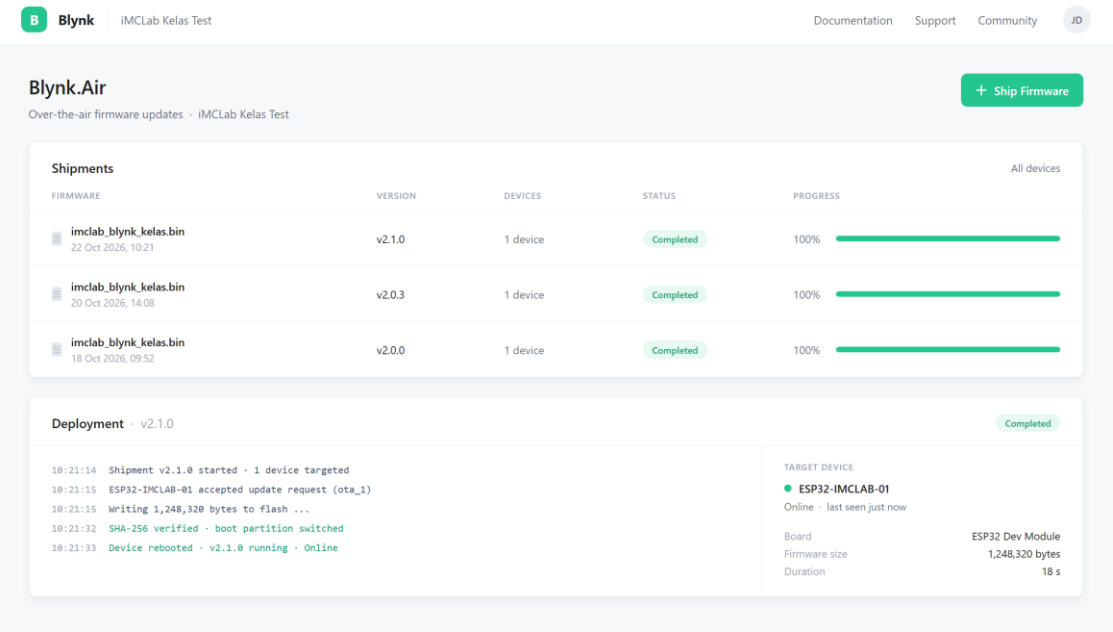
2. Unggah Firmware ke Blynk Cloud: Pada modul Blynk.Air di web dashboard, pengembang membuat deployment baru, menaikkan nomor versi firmware (misalnya dari v2.0 ke v2.1), dan mengunggah berkas .bin.

3. Notifikasi OTA ke Perangkat: Server Blynk mengirimkan sinyal pembaruan melalui Virtual Pin khusus atau mekanisme Blynk.Air. ESP32 merespons dengan membuka kanal HTTP stream.

4. Verifikasi dan Boot Ulang: Fungsi Update.writeStream() menulis data secara sekuensial ke partisi flash sekunder. Setelah fungsi Update.end() mengonfirmasi keberhasilan transfer, ESP.restart() dieksekusi untuk menjalankan firmware terbaru.

5.4 Pengujian Hasil Pembaruan Firmware OTA

Proses keberhasilan pengunggahan dan deployment nirkabel terekam pada konsol manajemen Blynk IoT:



Gambar 5.2 Halaman Blynk.Air pada Konsol Blynk IoT Menunjukkan Status Shipment Firmware v2.1.0 Selesai (Completed)

Analisis Hasil Uji: Firmware sebesar 1.248.320 byte berhasil ditransfer ke partisi ota_1 dalam waktu 18 detik pada jaringan WiFi lokal. Perangkat ESP32 melakukan warm-reboot secara otomatis dan terhubung kembali ke cloud Blynk dalam status operasional normal tanpa memerlukan intervensi kabel fisik.

KESIMPULAN DAN REFLEKSI RISET

Berdasarkan serangkaian eksperimen yang telah dilakukan terhadap platform robotika Kit iMCLab ESP32, dapat dirumuskan beberapa poin kesimpulan substantif:

1. Efektivitas Peripheral LEDC ESP32: Pemanfaatan timer hardware LEDC pada frekuensi 30 kHz terbukti mampu menghasilkan modulasi PWM yang halus, responsif, dan bebas gangguan bising akustik pada driver motor L298N.

2. Fleksibilitas Kontrol Python-Microcontroller: Penggunaan skrip Python dengan fitur auto-detect serial port mempermudah pengembangan sistem kendali robotika otonom, memungkinkan komputasi visi dan orkestrasi pergerakan dilakukan secara terpusat pada unit PC.

3. Keandalan Telemetri Blynk IoT: Integrasi platform Blynk berhasil menghadirkan instrumentasi lup tertutup yang efektif. Melalui interupsi perangkat keras optocoupler dengan penanganan anti-bouncing mikrodetik, pemantauan RPM motor dapat ditampilkan secara stabil dan real-time.

4. Keamanan dan Kemudahan Manajemen OTA: Arsitektur dual-partition pada ESP32 menjamin proses pembaruan perangkat tegar nirkabel berlangsung aman dan terproteksi dari risiko kegagalan sistem, memenuhi standar arsitektur robot cerdas industri.